

SIR Model on a Network

EdgeBasedModels.jl Vignette 01

Simon Frost

2026-03-30

Table of contents

Introduction	1
Setup	2
Defining the network	2
Building the compact SIR model	2
Solving the ODE	3
Expanded formulation	6
Effect of transmission rate	9
Summary	11

Introduction

Edge-based compartmental models (EBCMs) provide an exact, deterministic framework for tracking epidemics on configuration-model networks. Instead of simulating individual transmission events, we track the probability $\theta(t)$ that a random edge in the network has *not* transmitted infection by time t .

Combined with the network's probability generating function (PGF) $\psi(x) = \sum_k p_k x^k$ — which encodes the degree distribution — we can derive the fraction susceptible as $S(t) = \psi(\theta(t))$ and the fraction recovered $R(t)$ from a small system of ODEs.

The compact formulation (Miller 2011) for SIR on a configuration model network requires just two ODEs:

$$\begin{aligned}\dot{\theta} &= -\beta\theta + \beta\frac{\psi'(\theta)}{\psi'(1)} + \gamma(1 - \theta) \\ \dot{R} &= \gamma(1 - \psi(\theta) - R)\end{aligned}$$

where β is the per-edge transmission rate and γ is the recovery rate.

Setup

```
using EdgeBasedModels
using ModelingToolkit
using OrdinaryDiffEq
using Symbolics
using Plots
```

Defining the network

We model a Poisson (Erdős–Rényi) network with mean degree $\kappa = 5$. The PGF is $\psi(x) = e^{\kappa(x-1)}$.

```
@parameters β γ κ
pgf = poisson_pgf(κ)
```

```
DegreePGF(z, exp((-1 + z)*κ))
```

We can verify the mean degree:

```
md = mean_degree(pgf)
println("Mean degree: ", md)
```

```
Mean degree: exp(0)*κ
```

Building the compact SIR model

The `build_sir` function constructs the edge-based ODE system. The `:compact` form produces just 2 ODEs.

```
model_compact = build_sir(pgf, β, γ; form = :compact)
eqs = ModelingToolkit.equations(model_compact.system)
println("Number of ODEs: ", length(eqs))
for eq in eqs
    println(" ", eq)
end
```

Number of ODEs: 2

```
Differential(t, 1)(R(t)) ~ (1 - exp((-1 + θ(t))*κ) - R(t))*γ
Differential(t, 1)(θ(t)) ~ (-exp((-1 + θ(t))*κ)*β + θ(t)*exp(0)*β - (1 -
θ(t))*exp(0)*γ) / (-exp(0))
```

The model returns an `EdgeModelSystem` with the compiled system, a dictionary of state variables, and a dictionary of observables:

```
println("State variables: ", keys(model_compact.variables))
println("Observables:      ", keys(model_compact.observables))
```

```
State variables: [:R, :θ]
Observables:     [:I, :ψ_θ, :S]
```

Solving the ODE

We set $\beta = 0.6$, $\gamma = 0.1$, and $\kappa = 5$, and seed the epidemic with a small perturbation $\theta(0) = 1 - \varepsilon$.

```
tspan = (0.0, 80.0)
prob = ODEProblem(
    model_compact.system,
    merge(
        Dict(model_compact.variables[:θ] ⇒ 1 - 1e-3,
             model_compact.variables[:R] ⇒ 0.0),
        Dict(β ⇒ 0.6, γ ⇒ 0.1, κ ⇒ 5.0),
    ),
    tspan,
)
sol = solve(prob, Tsit5(); saveat = 0.5)
```

```
retcode: Success
Interpolation: 1st order linear
t: 161-element Vector{Float64}:
 0.0
 0.5
 1.0
 1.5
 2.0
 2.5
 3.0
```

```

3.5
4.0
4.5
?
76.0
76.5
77.0
77.5
78.0
78.5
79.0
79.5
80.0
u: 161-element Vector{Vector{Float64}}:
 [0.0, 0.999]
 [0.00045579397430321793, 0.9968638047100656]
 [0.0018371009216488273, 0.9903070663850259]
 [0.005859210659788087, 0.97128592939206]
 [0.01619273745353259, 0.9234804552451361]
 [0.03701964915146776, 0.8322933787287785]
 [0.0681560895029416, 0.7095982553176541]
 [0.10515742512836954, 0.5853898813577376]
 [0.14409823331591104, 0.47877777021808077]
 [0.18293279985537417, 0.3945054680538822]
 ?
 [0.9847109699474157, 0.1554485633342279]
 [0.984742121797434, 0.15546886700275334]
 [0.9847715051871666, 0.15546903015115954]
 [0.9847992662973192, 0.1554546289808019]
 [0.9848257121891636, 0.1554438643553165]
 [0.9848509069841553, 0.1554365736324455]
 [0.9848749059802897, 0.1554321506755919]
 [0.9848977559220498, 0.15542956652843512]
 [0.9849194950004069, 0.155427369414931]

```

Now we extract the population-level quantities using the model's named variables and observables rather than assuming a positional ordering in the ODE state vector. For a Poisson PGF, $S(t) = \psi(\theta(t)) = e^{\kappa(\theta-1)}$:

```

S_vals = compartment(sol, model_compact, :S)
I_vals = compartment(sol, model_compact, :I)
R_vals = compartment(sol, model_compact, :R)

```

161-element Vector{Float64}:

0.0
0.00045579397430321793
0.0018371009216488273
0.005859210659788087
0.01619273745353259
0.03701964915146776
0.0681560895029416
0.10515742512836954
0.14409823331591104
0.18293279985537417
⋮
0.9847109699474157
0.984742121797434
0.9847715051871666
0.9847992662973192
0.9848257121891636
0.9848509069841553
0.9848749059802897
0.9848977559220498
0.9849194950004069

```
plot(sol.t, S_vals, label = "S", linewidth = 2, color = :blue)
plot!(sol.t, I_vals, label = "I", linewidth = 2, color = :red)
plot!(sol.t, R_vals, label = "R", linewidth = 2, color = :green)
xlabel!("Time")
ylabel!("Fraction of population")
title!("SIR on Poisson network ( $\kappa=5$ ,  $\beta=0.6$ ,  $\gamma=0.1$ )")
```

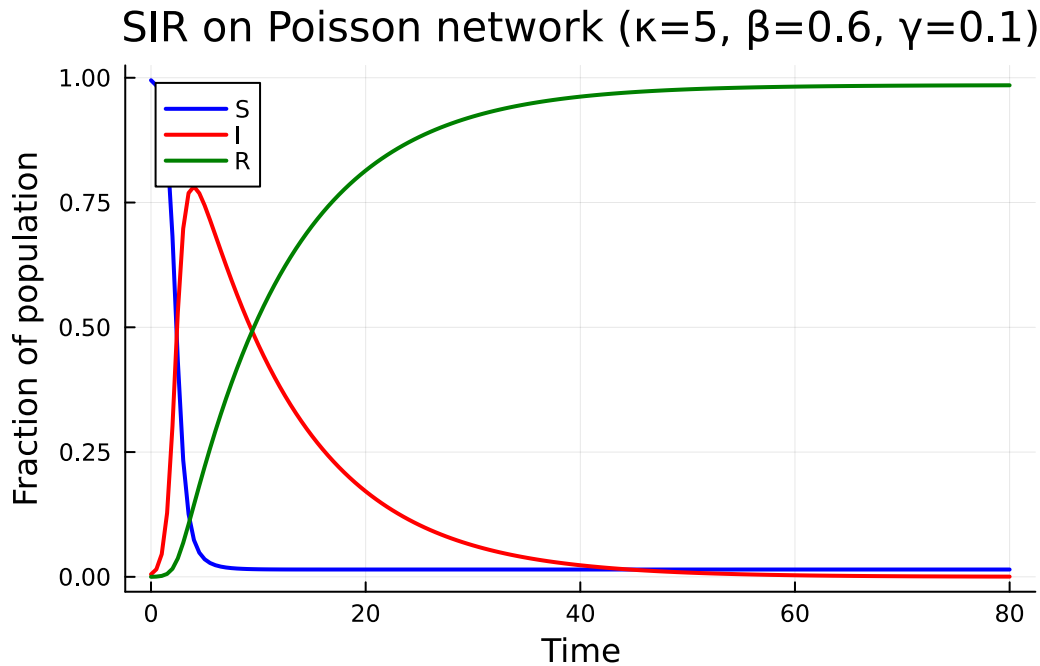


Figure 1: SIR epidemic on a Poisson network (compact formulation).

Expanded formulation

The expanded form tracks edge-level probabilities φ_I and φ_R explicitly, plus the population-level R . This form generalises to arbitrary disease progression graphs.

```
model_expanded = build_sir(pgf,  $\beta$ ,  $\gamma$ ; form = :expanded)
eqs_exp = ModelingToolkit.equations(model_expanded.system)
println("Number of ODEs: ", length(eqs_exp))
for eq in eqs_exp
    println(" ", eq)
end
```

Number of ODEs: 4

```
Differential(t, 1)(R(t)) ~ (1 - exp((-1 +  $\theta(t)$ )* $\kappa$ ) - R(t))* $\gamma$ 
Differential(t, 1)(phi_R(t)) ~ phi_I(t)* $\gamma$ 
Differential(t, 1)(phi_I(t)) ~ -phi_I(t)*( $\beta$  +  $\gamma$ ) + phi_S(t)*phi_I(t)* $\beta$ * $\kappa$ 
Differential(t, 1)( $\theta(t)$ ) ~ -phi_I(t)* $\beta$ 
```

```
println("State variables: ", keys(model_expanded.variables))
println("Observables:      ", keys(model_expanded.observables))
```

```
State variables: [:R, :φ_I, :φ_R, :θ]
Observables:     [:I, :φ_S, :S, :edge_hazard, :excess_hazard]
```

Let us solve the expanded model and verify it gives the same epidemic curve:

```
ic_exp = default_initial_conditions(model_expanded)
prob_exp = ODEProblem(
    model_expanded.system,
    merge(ic_exp, Dict{β ⇒ 0.6, γ ⇒ 0.1, κ ⇒ 5.0}),
    tspan,
)
sol_exp = solve(prob_exp, Tsit5(); saveat = 0.5)
```

```
retcode: Success
Interpolation: 1st order linear
t: 161-element Vector{Float64}:
 0.0
 0.5
 1.0
 1.5
 2.0
 2.5
 3.0
 3.5
 4.0
 4.5
 ⋮
76.0
76.5
77.0
77.5
78.0
78.5
79.0
79.5
80.0
u: 161-element Vector{Vector{Float64}}:
 [0.0, 0.0, 0.0, 0.999]
```

```
[0.00024324427816313003, 0.0, 0.0, 0.999]
[0.0004746253578775114, 0.0, 0.0, 0.999]
[0.0006947219791394167, 0.0, 0.0, 0.999]
[0.0009040847875264574, 0.0, 0.0, 0.999]
[0.0011032363037232105, 0.0, 0.0, 0.999]
[0.001292674673373526, 0.0, 0.0, 0.999]
[0.0014728742228724597, 0.0, 0.0, 0.999]
[0.001644286092156801, 0.0, 0.0, 0.999]
[0.0018073391791149339, 0.0, 0.0, 0.999]
```

?

```
[0.0049849791729324215, 0.0, 0.0, 0.999]
[0.0049850947331311745, 0.0, 0.0, 0.999]
[0.004985206513854202, 0.0, 0.0, 0.999]
[0.00498531531788702, 0.0, 0.0, 0.999]
[0.004985421858307163, 0.0, 0.0, 0.999]
[0.004985524225408373, 0.0, 0.0, 0.999]
[0.004985621599836726, 0.0, 0.0, 0.999]
[0.004985714225159776, 0.0, 0.0, 0.999]
[0.004985802333185553, 0.0, 0.0, 0.999]
```

```
S_exp = compartment(sol_exp, model_expanded, :S)
I_exp = compartment(sol_exp, model_expanded, :I)
```

161-element Vector{Float64}:

```
0.00498752080731768
0.0047442765291545504
0.0045128954494401685
0.004292798828178263
0.004083436019791223
0.00388428450359447
0.0036948461339441543
0.0035146465844452203
0.003343234715160879
0.0031801816282027466
```

?

```
2.5416343852587356e-6
2.426074186505714e-6
2.314293463478133e-6
2.205489430659978e-6
2.0989490105175965e-6
1.9965819093069673e-6
1.8992074809539433e-6
```

1.806582157904378e-6
1.718474132127168e-6

```
plot(sol.t, I_vals, label = "I (compact)", linewidth = 3, color = :red)
plot!(sol_exp.t, I_exp, label = "I (expanded)", linewidth = 2, color = :red,
      linestyle = :dash)
xlabel!("Time")
ylabel!("Fraction infected")
title!("Compact vs Expanded Formulation")
```

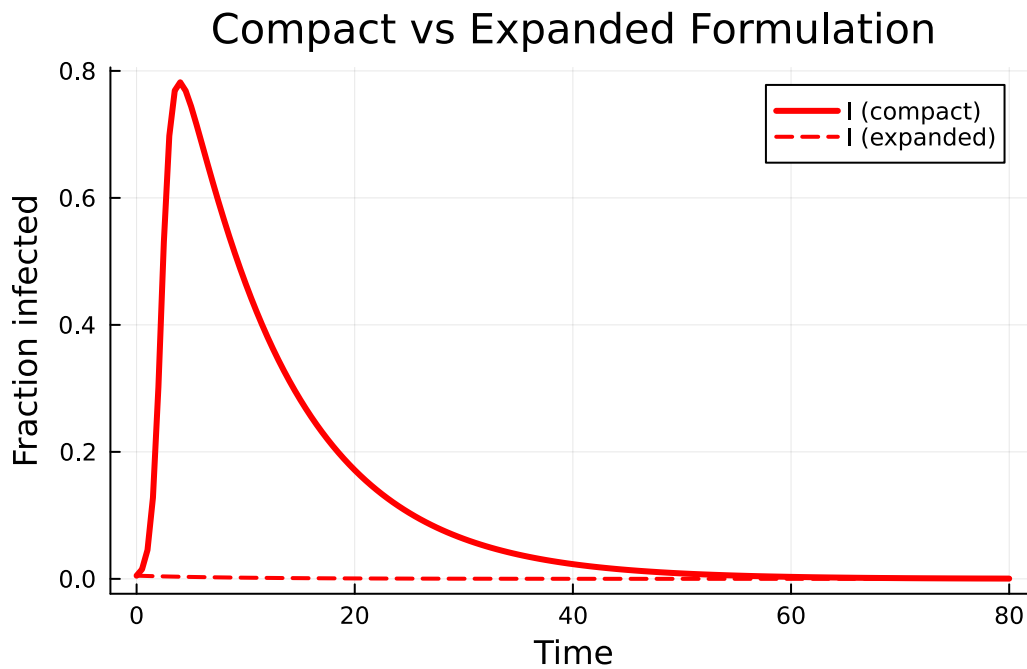


Figure 2: Compact (solid) and expanded (dashed) formulations give identical results.

The two formulations give identical epidemic curves, confirming that the compact form (2 ODEs) is an exact reduction of the expanded form (4 ODEs) for SIR.

Effect of transmission rate

```
betas = [0.2, 0.4, 0.6, 0.8]
p = plot(xlabel = "Time", ylabel = "Fraction infected",
        title = "Effect of  $\beta$  on epidemic dynamics ( $\kappa=5$ ,  $\gamma=0.1$ )")
```

```

for b in betas
    prob_b = ODEProblem(
        model_compact.system,
        merge(
            Dict(model_compact.variables[:θ] ⇒ 1 - 1e-3,
                model_compact.variables[:R] ⇒ 0.0),
            Dict(β ⇒ b, γ ⇒ 0.1, κ ⇒ 5.0),
        ),
        tspan,
    )
    sol_b = solve(prob_b, Tsit5(); saveat = 0.5)
    I_b = compartment(sol_b, model_compact, :I)
    plot!(p, sol_b.t, I_b, label = "β = $b", linewidth = 2)
end
p

```

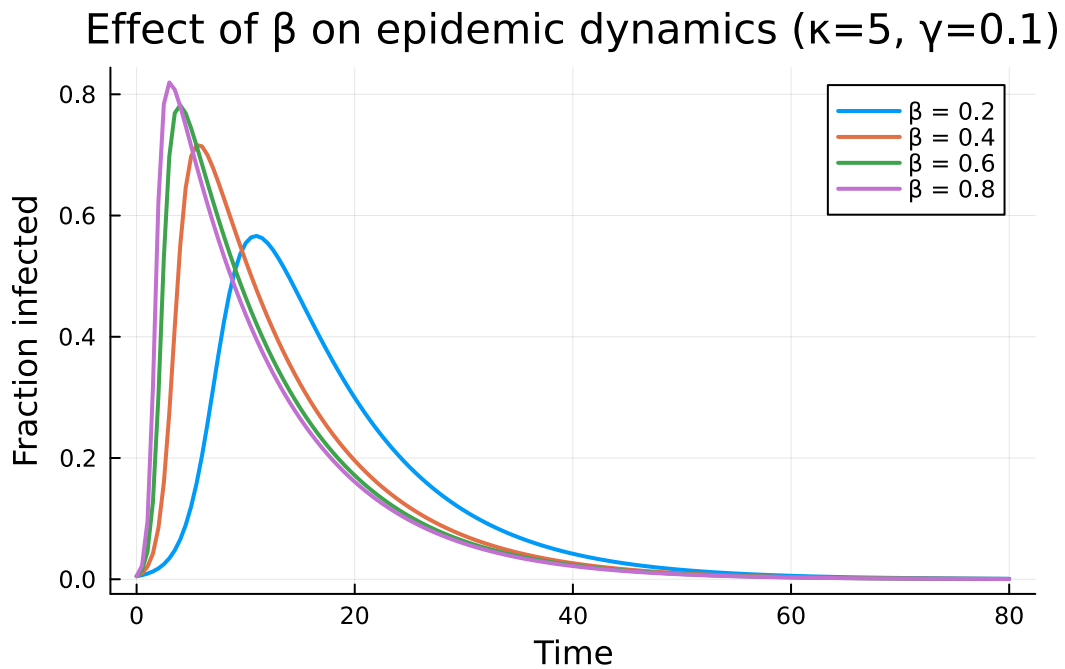


Figure 3: Effect of transmission rate β on the epidemic curve.

Summary

- The compact formulation produces 2 ODEs and is the most efficient for SIR on static networks.
- The expanded formulation produces 4 ODEs but generalises to arbitrary disease progression graphs (SEIR, multi-stage, etc.).
- Both formulations give identical results for SIR.
- The PGF encodes the network's degree distribution — we used a Poisson PGF here, but the package supports arbitrary degree distributions via `polynomial_pgf`.